

DQN Otonom Enerji Tasarrufu

Bu proje, ofis ortamlarında kullanılan HVAC/klima sistemlerinin enerji tüketimini azaltırken iç ortam konforunu koruyabilmesi için geliştirilmiş **Deep Q-Network (DQN)** tabanlı bir akıllı termostat simülasyonudur.

Projede ajan, dış sıcaklık, iç sıcaklık, gün içerisindeki zaman, yalıtım katsayısı ve ofisteki insan sayısı gibi çevresel durumları gözlemleyerek klimayı hangi güç modunda çalıştıracağına karar verir. Amaç, özellikle mesai saatlerinde iç ortam sıcaklığını **25°C hedef sıcaklığa** yakın tutarken gereksiz enerji tüketimini azaltmaktır.

İçindekiler

- [Proje Amacı](#)
- [Kullanılan Teknolojiler](#)
- [Proje Dosya Yapısı](#)
- [Markov Karar Süreci Modeli](#)
- [State Space](#)
- [Action Space](#)
- [Termodinamik Ortam Modeli](#)
- [Reward Function](#)
- [Deep Q-Network Mimarisi](#)
- [DQN Öğrenme Formülleri](#)
- [Hiperparametreler](#)
- [Kurulum](#)
- [Kullanım](#)
- [Görsel Çıktılar](#)
- [Sonuç ve Yorum](#)
- [Gelecek Geliştirmeler](#)

Proje Amacı

Geleneksel termostat sistemleri çoğunlukla sabit eşik değerlerine göre çalışır. Bu yaklaşım, bazı durumlarda gereksiz enerji tüketimine veya konfor kaybına neden olabilir. Bu projede ise termostat kontrolü, sabit kurallardan ziyade **pekiştirmeli öğrenme** yaklaşımıyla modellenmiştir.

Ajanın temel hedefleri:

- Mesai saatlerinde iç sıcaklığı **25°C** civarında tutmak
 - Mesai dışı saatlerde gereksiz klima kullanımını azaltmak
 - Farklı dış sıcaklık, oda büyüklüğü, yalıtım ve insan sayısı senaryolarına uyum sağlamak
 - Enerji tüketimi ile termal konfor arasında denge kurmak
 - Öğrenilmiş politika ile otonom klima kontrolü gerçekleştirmek
-

Kullanılan Teknolojiler

Teknoloji	Kullanım Amacı
Python	Ana programlama dili
PyTorch	DQN sinir ağı ve eğitim süreci
NumPy	Sayısal işlemler ve veri saklama
Matplotlib	Eğitim ve test grafiklerinin oluşturulması
Pillow	Dashboard görsellerinin çizimi
ImageIO	GIF çıktısı oluşturma
Deep Q-Network	Pekiştirmeli öğrenme algoritması
Experience Replay	Geçmiş deneyimlerden mini-batch öğrenme
Target Network	DQN eğitim kararlılığını artırma

Proje Dosya Yapısı

```
DQN-Otonom-Enerji-Tasarrufu/  
|  
├─ env.py                # Ofis ortamı, termodinamik model ve  
reward sistemi  
├─ agent.py              # DQN ağı, replay buffer ve ajan karar  
mekanizması  
├─ train.py              # Eğitim döngüsü  
├─ evaluate.py           # Eğitilmiş model ile 1 günlük test  
simülasyonu  
├─ plot_training.py      # Eğitim reward grafiği üretimi  
├─ gif_thermostat.py     # Dashboard GIF simülasyonu  
|  
├─ dqn_model.pth         # Eğitilmiş model ağırlıkları  
├─ reward_history.npy     # Episode bazlı reward geçmişi  
├─ dqn_egitim_grafigi.png # Eğitim grafiği çıktısı  
├─ dqn_test_sonuc_lari.png # Test simülasyonu grafiği  
├─ smart_thermostat_dashboard.gif # Dashboard simülasyon çıktısı  
|  
├─ requirements.txt      # Gerekli Python paketleri  
└─ README.md             # Proje dokümantasyonu
```

Markov Karar Süreci Modeli

Bu proje, ofis içi klima kontrol problemini bir **Markov Karar Süreci (MDP)** olarak ele alır.

Bir MDP genel olarak şu bileşenlerden oluşur:

$$MDP = \langle S, A, P, R, \gamma \rangle$$

Sembol	Açıklama
S	State space, yani ajan tarafından gözlemlenen durumlar
A	Action space, yani ajanın seçebileceği aksiyonlar
P	Transition model, yani bir durumdan diğerine geçiş dinamiği
R	Reward function, yani ajanın aldığı ödül/ceza
γ	Discount factor, yani gelecekteki ödüllerin bugünkü karara etkisi

Bu projede ajan her dakika ortamı gözlemler, bir klima modu seçer, ortam termodinamik modele göre güncellenir ve ajan seçtiği aksiyona göre ödül veya ceza alır.

State Space

Ajanın her zaman adımında aldığı durum vektörü 5 değişkenden oluşur:

$$s_t = [m_t, T_{\text{ext},t}, T_{\text{in},t}, U, N_t]$$

Değişken	Açıklama	Kod Karşılığı
m_t	Gün içerisindeki dakika değeri	<code>current_step</code>
$T_{\text{ext},t}$	Dış ortam sıcaklığı	<code>outside_temp</code>
$T_{\text{in},t}$	İç ortam sıcaklığı	<code>real_temp</code>
U	Odanın yalıtım katsayısı	<code>u_value</code>
N_t	Ofisteki insan sayısı	<code>num_humans</code>

Durum uzayı şu şekilde ifade edilebilir:

$$s_t \in \mathbb{R}^5$$

Kodda state vektörü PyTorch ile uyumlu olacak şekilde `float32` NumPy dizisi olarak döndürülür.

```
return np.array([
    self.current_step,
    self.outside_temp,
    self.real_temp,
    self.u_value,
    self.num_humans
], dtype=np.float32)
```

Action Space

Ajanın seçebileceği aksiyonlar ayrık bir action space olarak tanımlanmıştır:

$$A = \{0, 1, 2\}$$

Action	Klima Modu	Güç Değeri
0	Klima Kapalı	0 W
1	Bekleme Modu	2500 W
2	Performans Modu	5000 W

Aksiyon-güç eşlemesi:

$$P(a_t) = \begin{cases} 0, & a_t = 0 \\ 2500, & a_t = 1 \\ 5000, & a_t = 2 \end{cases}$$

Bu aksiyonlar üzerinden ajan, iç ortam sıcaklığına ve mesai durumuna göre enerji/konfor dengesini öğrenir.

Termodinamik Ortam Modeli

Ortam modeli, basitleştirilmiş bir ofis termodinamiği üzerine kuruludur. Her episode yeni bir günü temsil eder ve bir gün toplam **1440 dakika** olarak simüle edilir.

Ortam Başlangıç Değerleri

Her episode başında bazı değerler rastgele atanır:

Parametre	Değer Aralığı / Değer	Açıklama
Oda alanı	$A_{\text{room}} \sim U(10, 50)$ m ²	Her episode farklı oda büyüklüğü
Oda yüksekliği	2.8 m	Sabit tavan yüksekliği
Yalıtım katsayısı	$U \sim U(0.4, 0.9)$	Odanın ısı geçirgenliği
Dış sıcaklık	$T_{\text{ext}} \sim U(-10, 35)$ °C	Gün başlangıcı dış sıcaklığı
İnsan sayısı	0 veya 5-15	Mesai saatlerine göre değişir

Mesai Programı

Ofis doluluk durumu mesai saatlerine göre belirlenmiştir:

Zaman Aralığı	Durum
08:00 - 12:00	Aktif mesai

Zaman Aralığı	Durum
12:00 - 13:00	Öğle arası
13:00 - 17:00	Aktif mesai
Diğer saatler	Mesai dışı

Kodda dakika karşılığı:

$$W = [480, 720) \cup [780, 1020)$$

Mesai başlangıcından önce hazırlık yapılabilmesi için ayrıca iki hazırlık aralığı kullanılmıştır:

$$P_{\text{prep}} = [450, 480) \cup [750, 780)$$

Bu hazırlık aralıklarında ajan klimayı açarsa mesai dışı cezası almaz. Böylece ajan, mesai başlamadan önce ortamı hedef sıcaklığa yaklaştırmayı öğrenebilir.

Isı Geçiş Modeli

İç sıcaklık değişimi üç ana ısı bileşeniyle hesaplanır:

1. Dış ortamdan gelen/giden ısı etkisi
2. İnsanlardan kaynaklanan iç ısı yükü
3. Klimanın ısıtma veya soğutma etkisi

1. Isı Sızıntısı

Dış ortam ile iç ortam arasındaki sıcaklık farkından kaynaklanan ısı transferi:

$$\dot{Q}_{\text{leak}} = U \cdot A \cdot (T_{\text{ext},t} - T_{\text{in},t})$$

Burada:

Sembol	Açıklama
U	Yalıtım katsayısı
A_{room}	Oda alanı
$T_{\text{ext},t}$	Dış sıcaklık
$T_{\text{in},t}$	İç sıcaklık

2. İç Isı Yükü

Ofisteki her insanın yaklaşık 125 W ısı yaydığı varsayılmıştır:

$$\dot{Q}_{\text{internal}} = N_t \cdot 125$$

3. HVAC Etkisi

Klima, iç sıcaklığın hedef aralığa göre altında veya üstünde olmasına bağlı olarak ısıtma ya da soğutma etkisi üretir.

$$\dot{Q}_{AC} = \begin{cases} +P(a_t), & T < 24.5 \\ -P(a_t), & T_{in,t} > 25.5 \\ 0, & 24.5 \leq T_{in,t} \leq 25.5 \end{cases}$$

Burada $P(a_t)$ seçilen aksiyonun güç değeridir.

4. İç Sıcaklık Güncellemesi

Odanın termal kütlesi şu şekilde hesaplanır:

$$C_{room} = A_{room} \cdot h \cdot \rho_{air} \cdot c_{p,air}$$

Kodda kullanılan sabitler:

Parametre	Değer
h	2.8 m
ρ_{air}	1.2 kg/m ³
$c_{p,air}$	1005 J/kgK
Δt	60 saniye

İç sıcaklık değişimi:

$$\Delta T_{in} = \frac{(\dot{Q}_{leak} + \dot{Q}_{Q}) + \dot{Q}_{AC}}{C} \cdot \Delta t$$

Bir sonraki iç sıcaklık:

$$T_{in,t+1} = T_{in,t} + \Delta T_{in}$$

Reward Function

Ödül fonksiyonu, ajanın hem konforu sağlamasını hem de enerji tüketimini azaltmasını hedefler.

Toplam reward şu bileşenlerden oluşur:

$$R_t = R_{comf} + R_{occ} + R_{energy}$$

1. Mesai Saatlerinde Konfor Ödülü/Cezası

Mesai saatlerinde hedef sıcaklık:

$$T_{target} = 25^{\circ}C$$

Sıcaklık farkı:

$$d_t = |T_{\text{target}} - T_{\text{in},t}|$$

Mesai saatlerinde konfor reward'u:

$$R_{\text{comfort}} = \begin{cases} +20, & d_t \leq 1 \\ -2 \cdot d_t^2, & d_t > 1 \end{cases}$$

Bu yapı sayesinde hedef sıcaklığa yakın durumlar ödüllendirilirken, hedef sıcaklıktan uzaklaşma karesel olarak cezalandırılır. Karesel ceza, büyük sıcaklık sapmalarını daha sert cezalandırır.

Örnek:

Sıcaklık Farkı	Ceza
1°C	Konfor aralığında, +20
2°C	$-2 \cdot 2^2 = -8$
3°C	$-2 \cdot 3^2 = -18$
5°C	$-2 \cdot 5^2 = -50$

2. Mesai Dışı Gereksiz Kullanım Cezası

Mesai dışı saatlerde ve hazırlık aralığı dışında klima açılırsa ajan ceza alır:

$$R_{\text{occupancy}} = \begin{cases} -10, & t \notin W, t \notin P_{\text{prep}}, a_t > 0 \\ 0, & \text{diğer durumlar} \end{cases}$$

Bu sayede ajan, ofis boşken klimayı gereksiz yere çalıştırmamayı öğrenir.

3. Enerji Tüketim Cezası

Klima gücü arttıkça enerji cezası da artar:

$$R_{\text{energy}} = \begin{cases} 0, & a_t = 0 \\ -1, & a_t = 1 \\ -3, & a_t = 2 \end{cases}$$

Bu ceza, ajanın sürekli performans modunda çalışmasını engeller. Ajan, hedef sıcaklığa yaklaşıncaya daha düşük enerji modlarını veya kapalı modu tercih etmeye yönlendirilir.

Reward Fonksiyonunun Genel Yorumu

Bu reward tasarımıyla ajan şu davranışları öğrenir:

- Mesai saatlerinde sıcaklığı 25°C civarında tutmak
- Büyük sıcaklık sapmalarını hızlı şekilde düzeltmek
- Mesai dışı gereksiz klima kullanımından kaçınmak
- Hazırlık aralığında mesaiye önceden hazırlanmak
- Performans modunu yalnızca gerektiğinde kullanmak

Deep Q-Network Mimarisi

Projede kullanılan DQN modeli, tam bağlantılı katmanlardan oluşan basit ama etkili bir yapay sinir ağıdır.

Model mimarisi:

Input Layer	: 5 nöron
Hidden Layer 1	: 64 nöron + ReLU
Hidden Layer 2	: 64 nöron + ReLU
Output Layer	: 3 nöron

Ağ yapısı matematiksel olarak şu şekilde ifade edilebilir:

$$h_1 = \text{ReLU}(W_1 s_t + b_1)$$

$$h_2 = \text{ReLU}(W_2 h_1 + b_2)$$

$$Q(s_t, a; \theta) = W_3 h_2 + b_3$$

Çıkış katmanında 3 değer üretilir:

$$Q(s_t, a_0), Q(s_t, a_1), Q(s_t, a_2)$$

Bu değerler, ilgili aksiyonların beklenen uzun vadeli toplam ödülünü temsil eder. Ajan, keşif yapmadığı durumlarda en yüksek Q-değerine sahip aksiyonu seçer.

DQN Öğrenme Formülleri

Q-Learning Güncelleme Mantığı

Q-learning'in temel amacı, her durum-aksiyon çifti için beklenen toplam ödülü öğrenmektir.

Klasik Q-learning hedefi:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

DQN'de tablo yerine sinir ağı kullanılır:

$$Q(s, a; \theta)$$

Burada θ , sinir ağının öğrenilebilir ağırlıklarıdır.

Bellman Target

DQN eğitiminde hedef Q-değeri şu şekilde hesaplanır:

$$y_t = r_t + \gamma \max_{a'} Q_{\text{target}}(s_{t+1}, a'; \theta^-) \cdot (1 - \text{done})$$

Burada:

Sembol	Açıklama
r_t	Anlık reward
γ	Discount factor
Q_{target}	Target network tarafından üretilen Q-değeri
θ^-	Target network ağırlıkları
done	Episode bitiş durumu

Episode bittiyse gelecek ödül hesaba katılmaz.

Loss Function

Policy network tarafından üretilen Q-değeri:

$$Q_{\text{policy}}(s_t, a_t; \theta)$$

Loss fonksiyonu Mean Squared Error olarak hesaplanır:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \left(y_i - Q_{\text{policy}}(s_i, a_i; \theta) \right)^2$$

Burada N , mini-batch boyutudur.

Target Network

DQN eğitiminde kararlılığı artırmak için iki ayrı ağ kullanılır:

Ağ	Görev
Policy Network	Güncel aksiyon değerlerini öğrenir
Target Network	Bellman hedefini daha stabil hesaplar

Target network, belirli aralıklarla policy network ağırlıklarıyla güncellenir:

$$\theta^- \leftarrow \theta$$

Bu projede target network her **10 episode** sonunda güncellenir.

Experience Replay

Ajanın deneyimleri replay buffer içinde saklanır:

$$D = \{(s_t, a_t, r_t, s_{t+1}, \text{done}_t)\}$$

Eğitim sırasında bu hafızadan rastgele mini-batch örnekleri çekilir. Bu yöntem:

- Ardışık veriler arasındaki korelasyonu azaltır
- Öğrenmeyi daha stabil hale getirir
- Önceki deneyimlerin tekrar kullanılmasını sağlar

Replay buffer kapasitesi:

$$|D|_{\max} = 10000$$

Aksiyon Seçimi: Epsilon-Greedy Policy

Ajan, eğitim sırasında keşif ve sömürü dengesini epsilon-greedy stratejisi ile kurar.

$$a_t = \begin{cases} \text{random action}, & \text{with probability } \epsilon \\ \arg\max_a Q(s_t, a; \theta), & \text{with probability } 1 - \epsilon \end{cases}$$

Başlangıçta ajan daha fazla rastgele aksiyon seçer. Eğitim ilerledikçe epsilon değeri azalır ve ajan öğrendiği politikaya daha fazla güvenmeye başlar.

Epsilon azalımı:

$$\epsilon_{\text{episode}+1} = \epsilon_{\text{episode}} \cdot 0.995$$

Alt sınır:

$$\epsilon_{\min} = 0.01$$

500 episode sonunda epsilon yaklaşık olarak şu seviyeye iner:

$$\epsilon_{500} \approx 1.0 \cdot 0.995^{500} \approx 0.082$$

Hiperparametreler

Eğitim Hiperparametreleri

Parametre	Değer	Açıklama
Episode sayısı	500	Ajanın 500 farklı gün senaryosunda eğitilmesi
Max step	1440	Her episode 1 gün, yani 1440 dakika
Batch size	64	Replay buffer'dan çekilen mini-batch boyutu
Discount factor γ	0.99	Gelecekteki ödüllerin önemi
Learning rate	0.001	Adam optimizer öğrenme hızı
Optimizer	Adam	Sinir ağı ağırlık güncellemesi

Parametre	Değer	Açıklama
Loss function	MSELoss	Bellman hedefi ile tahmin farkı
Replay buffer capacity	10000	Saklanan maksimum deneyim sayısı
Target update frequency	10 episode	Target network güncelleme aralığı
Initial epsilon	1.0	Başlangıç keşif oranı
Minimum epsilon	0.01	Minimum keşif oranı
Epsilon decay	0.995	Episode sonunda epsilon çarpanı

Model Hiperparametreleri

Katman	Giriş	Çıkış	Aktivasyon
Linear 1	5	64	ReLU
Linear 2	64	64	ReLU
Linear 3	64	3	Yok

Ortam Parametreleri

Parametre	Değer	Açıklama
Hedef sıcaklık	25°C	Mesai saatlerinde hedeflenen iç sıcaklık
Konfor aralığı	24°C - 26°C	Hedef sıcaklığa yakın kabul edilen aralık
Hazırlık aralıkları	07:30-08:00, 12:30-13:00	Mesai öncesi hazırlık imkanı
Mesai saatleri	08:00-12:00, 13:00-17:00	Konforun öncelikli olduğu saatler
İnsan başına ısı	125 W	İç ısı yükü
Klima kapalı	0 W	Action 0
Bekleme modu	2500 W	Action 1
Performans modu	5000 W	Action 2
Hava yoğunluğu	1.2 kg/m ³	Termal kütle hesabı
Hava özgül ısı	1005 J/kgK	Termal kütle hesabı

Kurulum

Projeyi klonlayın:

```
git clone https://github.com/KadirsinasCelik/DQN-0tonom-Enerji-Tasarrufu.git
cd DQN-0tonom-Enerji-Tasarrufu
```

Sanal ortam oluşturun:

```
python -m venv venv
```

Windows için sanal ortamı aktif edin:

```
venv\Scripts\activate
```

Linux/macOS için:

```
source venv/bin/activate
```

Gerekli kütüphaneleri kurun:

```
pip install -r requirements.txt
```

requirements.txt içeriği:

```
torch>=2.0.0  
numpy>=1.21.0  
matplotlib>=3.5.0  
imageio>=2.31.0  
pillow>=10.0.0
```

Kullanım

1. Modeli Eğitme

```
python train.py
```

Eğitim tamamlandığında şu dosyalar oluşur:

```
dqn_model.pth  
reward_history.npy
```

2. Eğitim Grafiği Oluşturma

```
python plot_training.py
```

Çıktı:

dqn_egitim_grafigi.png

3. Eğitilmiş Modeli Test Etme

python evaluate.py

Çıktı:

dqn_test_sonuclari.png

4. Dashboard GIF Üretme

python gif_thermostat.py

Çıktı:

smart_thermostat_dashboard.gif

Görsel Çıktılar

Eğitim Eğrisi

Ajanın 500 episode boyunca aldığı toplam reward değerleri ve hareketli ortalama eğrisi:

Eğitim Eğrisi

1 Günlük Test Simülasyonu

Eğitilmiş ajanın 1 günlük test senaryosunda iç sıcaklık, dış sıcaklık, insan sayısı ve klima aksiyonlarını gösteren grafik:

Test Sonuçları

Dashboard Simülasyonu

Ajanın kararlarını görsel olarak sunan dashboard GIF çıktısı:

Dashboard

Sonuç ve Yorum

Eğitim sürecinde ajan başlangıçta yüksek keşif oranı nedeniyle rastgele klima kararları verir. Episode sayısı arttıkça epsilon değeri azalır ve ajan daha bilinçli kararlar üretmeye başlar.

Modelin öğrendiği temel davranışlar:

- Mesai saatlerinde iç sıcaklığı hedef değer olan 25°C civarında tutmak
- Hedef sıcaklıktan sapma büyüdüğünde daha güçlü klima modunu seçmek
- Hedef sıcaklığa yaklaştığında enerji tasarrufu için daha düşük güç veya kapalı mod tercih etmek
- Mesai dışı saatlerde gereksiz klima kullanımını azaltmak
- Hazırlık saatlerinde ortamı mesaiye önceden hazırlamak

Örnek test çalıştırmasında model yaklaşık **6700+ toplam reward** seviyesine ulaşmıştır. Ortam başlangıç değerleri rastgele üretildiği için test reward değeri her çalıştırmada bir miktar değişebilir.

CPU/GPU Uyumluluk Notu

Eğer model GPU üzerinde eğitilip CPU kullanılan bir bilgisayarda test edilecekse, model yükleme sırasında `map_location` kullanılması önerilir.

Önerilen kullanım:

```
agent.policy_net.load_state_dict(  
    torch.load("dqn_model.pth", weights_only=True, map_location=agent.device)  
)
```

Bu düzenleme özellikle `evaluate.py` ve `gif_thermostat.py` dosyalarında model yükleme hatalarını önlemeye yardımcı olur.

Gelecek Geliştirmeler

Projeye ileride şu geliştirmeler eklenebilir:

- State normalizasyonu uygulanabilir
- Double DQN mimarisi eklenebilir
- Dueling DQN denenebilir
- Prioritized Experience Replay kullanılabilir
- Eğitim checkpoint sistemi eklenebilir
- Enerji tüketimi kWh cinsinden ayrıca hesaplanabilir
- Farklı bina tipleri ve farklı yalıtım senaryoları karşılaştırılabilir
- Web tabanlı dashboard arayüzü geliştirilebilir
- Gerçek sıcaklık sensörü verileriyle model test edilebilir

Geliřtirici

Kadir elik

Bu proje, derin pekiřtirmeli ğrenme kullanılarak akıllı bina otomasyonu ve enerji tasarrufu alanında rnek bir otonom kontrol sistemi geliřtirmek amacıyla hazırlanmıřtır.